CPSC538 Final Report

Kohtaro Uchida, Shuo Zhang, Zihan Pan

December 2, 2025

1 Base Algorithm (Route Selection, Pheromone Updates)

1.1 Common Parameters

- P'_{kn}: The probability of selecting neighbor node n at node k toward destination d
- P_{kn}: The routing table probability for neighbor n at node k
- N_k: The set of neighbors of node k

1.2 AntNet

1.2.1 Route Selection

The probability of selecting neighbor node n at node k toward destination d:

$$P'_{kn} = \frac{P_{kn} + \alpha \cdot l_n}{1 + \alpha(|N_k| - 1)} \tag{1}$$

where:

- α: weight coefficient (0.2–0.5)
- l_n: heuristic correction factor based on queue length

Heuristic correction factor:

$$l_n = 1 - \frac{q_n}{\sum_{n'=1}^{|N_k|} q_{n'}} \tag{2}$$

where q_n is the queue length (bits) at link to node n.

1.2.2 Pheromone Updates

Positive reinforcement (for selected link f):

$$P_{kf} \leftarrow P_{kf} + r(1 - P_{kf}) \tag{3}$$

Negative reinforcement (for other links $n \neq f$):

$$P_{kn} \leftarrow P_{kn} - r \cdot P_{kf}, \quad n \in N_k,\, n \neq f \tag{4}$$

where:

- r: reinforcement value ($0 < r \leq 1$) based on path quality
- f: previous node in the forward path

Reinforcement Value Calculation The reinforcement value r is computed based on the ant's trip time T and local statistical model $\mathcal{M}$:

$$r = c_1 \left(\frac{W_{best}}{T} \right) + c_2 \left(\frac{I_{sup} - I_{inf}}{(I_{sup} - I_{inf}) + (T - I_{inf})} \right) \tag{5}$$

where:

- T: observed trip time from node k to destination d
- W_{best}: best (minimum) trip time in recent observation window
- I_{inf}: lower bound of confidence interval ($= W_{best}$)
- I_{sup}: upper bound of confidence interval ($= \mu_d + z(\sigma_d/\sqrt{|W|})$)
- μ_d: exponential moving average of trip times
- σ_d: standard deviation of trip times
- z: confidence level coefficient (≈ 1.70 for 75–80% confidence)
- $|W|$: observation window size
- c_1, c_2: weight coefficients ($c_1 = 0.7$, $c_2 = 0.3$)

Squash function transformation:

$$r' = \frac{s(r)}{s(1)}, \quad s(x) = \left(1 + \exp\left(-\frac{a}{x|N_k|} \right) \right)^{-1} \tag{6}$$

Local Statistical Model Each node k maintains statistics $\mathcal{M}_k(d)$ for each destination d, updated via exponential moving averages:

$$\mu_d \leftarrow \mu_d + \eta(o_{k\to d} - \mu_d) \tag{7}$$

$$\sigma_d^2 \leftarrow \sigma_d^2 + \eta((o_{k\to d} - \mu_d)^2 - \sigma_d^2) \tag{8}$$

where $\eta = 0.005$ is the learning rate and $o_{k\to d}$ is the newly observed trip time.

Additionally, a sliding observation window W_d maintains recent samples, with $W_{best,d} = \min(W_d)$ and maximum size $|W|_{max} = 5(c/\eta)$ where $c = 0.3$.

Subpath When a backward ant arrives at node k from neighbor node f, it updates routing information not only for the original destination d, but also for intermediate nodes on the path under certain conditions.

Let $S_{k\to d}$ denote the set of nodes visited by the forward ant after leaving node k. The backward ant considers:

- Primary destination d: Always updated.
- Sub-destinations $d' \in S_{k\to d}, d' \neq d$:
  Updated only if:
  $$T_{k\to d'} < \mu_{d'} + I(\mu_{d'}, \sigma_{d'}) \tag{9}$$
  where:
  - $T_{k\to d'}$: measured trip time from node k to sub-destination d'
  - $\mu_{d'}$: exponential moving average of trip times to d'
  - $\sigma_{d'}$: standard deviation estimate of trip times to d'
  - $I(\mu, \sigma)$: confidence interval estimate

1.2.3 Forward Ants Configuration

From each network node s, a forward ant $F_{s\to d}$ is launched at regular intervals Δt toward a destination node d. Forward ants share the same queues as data packets, thereby experiencing identical traffic loads.

The destination node d is selected with probability p_d based on local data traffic patterns:

$$p_d = \frac{f_{sd}}{\sum_{d'=1}^{N} f_{sd'}} \tag{10}$$

where:

- f_{sd}: data flow from node s to node d (measured in bits or packets)
- N: total number of nodes in the network

This mechanism enables ants to adapt their exploration activity to the varying data traffic distribution.

1.3 Fault Tolerance (Without Backbone)

1.3.1 Route Selection

$$P'_{kn} = \frac{[P_{kn}]^{\alpha} \cdot \left(\frac{1}{h_n}\right)^{\beta}}{\sum_{n'=1}^{|N_k|} [P_{kn'}]^{\alpha} \cdot \left(\frac{1}{h_{n'}}\right)^{\beta}}, \quad \forall n \in N_k, n \neq \text{prev} \tag{11}$$

where:

- $\alpha = 8$: pheromone weight coefficient
- h_n: minimum hop count from node n to the destination d
- $\beta = 6$: hop count weight
- prev: previous node (excluded to prevent backtracking)

1.3.2 Pheromone Updates

Pheromone increment (without backbone):

$$\Delta P_t = \Delta P_{t-1} + \frac{A}{l_r} + C \tag{12}$$

Pheromone evaporation and update:

$$P_t = (\rho \cdot P_{t-1}) + \Delta P_t \tag{13}$$

where:

- P_t: pheromone value at time t
- ΔP_t: pheromone change at time t
- l_r: total route length (including paths to/from backbone)
- $A = 1$: route length weight
- $C = 1$: number of ants weight
- $\rho = 0.9$: evaporation rate

1.4 AntoHocNet

1.4.1 Specific Parameters

1.4.2 Route Selection

For reactive forward ants:

$$P'_{kn} = \frac{(P_{kn})^{\beta_1}}{\sum_{n'=1}^{|N_k|} (P_{kn'})^{\beta_1}} \tag{14}$$

For proactive forward ants:

if $\exists P_{kn}$ (pheromone information exists):

$$\text{Action} = \begin{cases} \text{Unicast } P'_{kn} = \frac{(P_{kn})^{\beta_1}}{\sum_{n'=1}^{|N_k|}(P_{kn'})^{\beta_1}} & \\ \text{Broadcast to all } n' \in N_k, & \text{if } b_{count} < n_b \end{cases} \tag{15}$$

else (no pheromone information):

$$\text{Action} = \begin{cases} \text{Broadcast to all } n' \in N_k, & \text{if } b_{count} < n_b \\ \text{Delete ant}, & \text{if } b_{count} \geq n_b \end{cases} \tag{16}$$

For data packets:

$$P'_{kn} = \frac{(P_{kn})^{\beta_2}}{\sum_{n'=1}^{|N_k|}(P_{kn'})^{\beta_2}}, \quad \beta_2 = 2 \tag{17}$$

where:

- β_1, β_2: exponent parameters ($\beta_2 > \beta_1 \geq 1$)
- b_{count}: the current broadcast counter for this ant
- $n_b = 2$: the maximum allowed broadcasts

1.4.3 Pheromone Updates

Quality measure calculation:

$$\tau_d^k = \left(\frac{\hat{T}_d^k + h \cdot T_{hop}}{2} \right)^{-1} \tag{18}$$

Exponential moving average update:

$$P_{kn} = \gamma \cdot P_{kn} + (1 - \gamma) \cdot \tau_d^k \tag{19}$$

where:

- τ_d^i: quality value (inverse of cost) for path to d from node i
- $\hat{T}_d^i$: estimated travel time to destination d
- h: number of hops to destination
- T_{hop}: fixed time per hop in unloaded conditions
- $\gamma = 0.7$: smoothing parameter for moving average

2 Additional Features

2.1 Beacon Ants (Fault Tolerance)

2.1.1 Beacon Counter Management

Each node k maintains:

- B_k^{sent}: number of beacons sent by node k
- B_{kn}^{recv}: number of beacons received from neighbor $n \in N_k$

On beacon transmission:

$$B_k^{sent} \leftarrow B_k^{sent} + 1 \tag{20}$$

On beacon reception from neighbor n:

$$B_{kn}^{recv} \leftarrow B_{kn}^{recv} + 1 \tag{21}$$

2.1.2 Sliding Window Mechanism

For each neighbor $n \in N_k$, node k maintains a sliding window W_{kn} of size w:

$$W_{kn} = [w_1, w_2, \ldots, w_w] \tag{22}$$

Window update (on beacon transmission):

$$W_{kn} \leftarrow [B_{kn}^{recv}, w_1, w_2, \ldots, w_{w-1}] \tag{23}$$

2.1.3 Fault Detection

Consecutive same-value count:

$$q_{kn} = \max\{j \mid w_1 = w_2 = \cdots = w_j,\ 1 \leq j \leq w\} \tag{24}$$

Fault evaporation rate:

$$\rho_{ft}^{kn} = D - \frac{q_{kn}}{w} \tag{25}$$

where:

- $D = 1.35$: base constant
- $w = 10$: window size
- q_{kn}: consecutive count (indicates stagnation)

Fault detection condition:

$$\text{Fault Detected} \iff \rho_{ft}^{kn} < 1 \iff q_{kn} > (D-1) \cdot w \tag{26}$$

Pheromone reduction (Beacon-based):

$$P_{kn} \leftarrow \rho_{ft}^{kn} \cdot P_{kn} \tag{27}$$

2.1.4 Runicast-based Fault Detection

Pheromone reduction (on reliable unicast timeout):

$$P_{kn} \leftarrow \rho_{rft} \cdot P_{kn} \tag{28}$$

where $\rho_{rft} = 0.2$ is the fixed evaporation rate for Runicast failures.

2.1.5 Beacon Transmission Schedule

Beacons are broadcast:

- At the start of each cycle
- At the end of each cycle

2.1.6 Integrated Pheromone Update

Combined update incorporating normal evaporation, backward ant reinforcement, and fault detection:

$$P_{kn}^{new} = (\rho \cdot P_{kn}^{old}) + \Delta P_t - F_{kn} \tag{29}$$

where:

$$F_{kn} = \begin{cases} (1 - \rho_{ft}^{kn}) \cdot P_{kn}^{old}, & \text{if Beacon fault detected} \\ (1 - \rho_{rft}) \cdot P_{kn}^{old}, & \text{if Runicast timeout} \\ 0, & \text{otherwise} \end{cases} \tag{30}$$

2.1.7 Performance Metrics

Detection time (cycles from failure to detection):

$$T_{detect} = \min\{t \mid q_{kn}(t) \geq (D-1) \cdot w\} \tag{31}$$

Recovery time (cycles from detection to successful delivery):

$$T_{recover} = t_{success} - t_{detect} \tag{32}$$

where $t_{success}$ is the first cycle with successful backbone-based delivery to all sinks.

2.2 Hello Message and Route Repair Ant (AntoHocNet)

2.2.1 Hello Message Mechanism

Hello messages are periodically broadcast to maintain neighbor awareness and enable pheromone diffusion:

$$\text{Hello broadcast interval: } t_{hello} = 1 \text{ second} \tag{33}$$

Neighbor management:

- Add sender n as destination in routing table upon hello reception
- Set initial pheromone: $P_{kn} = P_{init}$
- Remove neighbor after missing $allowed\_hello\_loss = 2$ consecutive hellos

Link failure detection time:

$$t_{failure} = t_{hello} \times allowed\_hello\_loss = 2 \text{ seconds} \tag{34}$$

2.2.2 Route Repair Mechanism

When a link failure is detected during data transmission and no alternative path exists:

Route Repair Ant broadcasting:

$$\text{Repair ant behavior} = \begin{cases} \text{Follow pheromone}, & \text{if } \exists P_{kn} \\ \text{Broadcast}, & \text{if } P_{kn} \text{ and } b_{repair} < 2 \\ \text{Delete}, & \text{if } b_{repair} \geq 2 \end{cases} \tag{35}$$

where b_{repair} is the broadcast counter for repair ants (maximum = 2).

Repair timeout:

$$t_{wait} = 5 \times \hat{T}_{lost} \tag{36}$$

where $\hat{T}_{lost}$ is the estimated end-to-end delay of the lost path.

2.2.3 Link Failure Notification

Upon link failure, nodes broadcast notifications containing:

- List of destinations with lost best paths
- New best estimates $(delay, hops)$ for affected destinations
- Propagation continues if neighbors lose their best/only path

Pheromone table update upon failure:

$$P_{kn} = \begin{cases} 0, & \text{if } n = \text{failed\_neighbor} \\ P_{kn}, & \text{otherwise} \end{cases} \tag{37}$$

This combination of hello messages and repair mechanisms ensures rapid failure detection and recovery while maintaining network connectivity.